Code review for Drafter requirements

Share

can you check my code for any bugs
also these are the requirements, let me know if they meet them: At least 7 routes.

- At least 5 Pages, or 10 FragmentsLinks to an external site., or a combination thereof.
- The implementation of the State dataclass must have at least 4 fields.
  - At least one field must be a list of class instances or a dictionary of class instances.
    - This means that there must be a second dataclass.
  - At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.
- At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.
- At least 3 meaningful if statements.
- At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.
- The site should have a legitimate purpose or functionality.
- No global variables (global constantsLinks to an external site. are fine).
- You must use the Drafter library.
- Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

Show more

Your code is in good shape overall. I found a few real bugs and a couple of requirement concerns.

Bugs

1. Error messages show up on the wrong page (most important).
`state.message` is only displayed on the home page. When `save_project` fails validation, it sets the message and returns `add_project(state)`, but `add_project` never displays `state.message`. The user clicks Save, nothing visibly happens, and the error then appears later on the home page. The same problem exists in `save_task`, `save_material`, and `update_task_status`. The fix is to show the message on each page, e.g. add `state.message` to the content in `add_project`, `add_task`, `add_material`, `projects`, `tasks`, `materials`.

2. Messages never clear.
Because the message is never reset, a stale message like "Project successfully added!" will keep appearing on the home page forever. Reset it at the start of a route, or after displaying it (`state.message = ""`).

3. Duplicate project names break things.
Everything links tasks and materials to projects by name. If two projects share a name, tasks, materials, progress, and costs get mixed together, and `project_details` just picks the last match. Add a check in `save_project`:

```python
for project in state.projects:
    if project.name == project_name:
        state.message = "A project with that name already exists."
        return add_project(state)
```

4. "Add a Task" / "Add Materials" from the project details page ignore which project you were viewing.
The dropdown always defaults to the first project. You have `state.selected_project` in your State but never use it. Set it in `project_details` (`state.selected_project = project_name`) and use it as the SelectBox default in `add_task` and `add_material`. That also fixes the requirement about modified fields (see below).

5. Float display issues.
`str(material.amount * material.price)` can show things like `$30.299999999999997`. Use `round(value, 2)` or `f"{value:.2f}"` everywhere you show money.

6. Invalid number input crashes or errors.
`project_budget`, `amount`, and `price` are `float` parameters filled from TextBoxes. If the user types "abc" or leaves it blank, Drafter can't convert it and will show an error page instead of your friendly message. Depending on your Drafter version, you may want to take these as `str` and convert them yourself with a `try/except`. If you do, document it, since try/except may count as an advanced technique for your course.

7. Unused field.
`Project.completion_percentage` is updated but never read (you recompute progress with `calculate_progress` each time). Either display that field or remove it. It's not a crash, just redundant.

Requirements check

Requirement	Status
At least 7 routes	✅ Plenty (~16)
At least 5 pages	✅
State has at least 4 fields	✅ (5 fields)
List of class instances, with a second dataclass	✅ `projects`, `tasks`, `materials`
4 state fields meaningfully modified in routes	⚠️ **Currently only 3** modified: `projects`, `tasks`, `materials`, and `message`. `message` is modified, but it's weak and arguably doesn't count as meaningful. `selected_project` is **never changed**, so it's effectively a constant. Fix bug #4 above and you'll clearly have 4 (or more).
At least 3 input fields	✅ TextBox, TextArea, SelectBox
At least 3 meaningful ifs	✅ (validation, status checks, etc.)
Loop over an attribute of the list of class instances	✅ e.g. `task.project_name`, `material.amount * material.price`
Legitimate purpose	✅
No global variables	✅ `TASK_STATUSES` and `MATERIAL_UNITS` are constants, which is allowed
Uses Drafter	✅
Advanced techniques documented	⚠️ Mostly none used. Type hints on `list[Project]` are fine, but if you add `try/except` or `f"{x:.2f}"`, add comments explaining them.

Summary

The only requirement at real risk is the **4 modified state fields**, which is fixed by actually using `selected_project`. The most noticeable functional bug is that error and success messages don't appear where the user expects them. Fix those two and handle the duplicate-name issue, and you should be in good shape.

I can write out the corrected versions of the affected routes if you'd like.

can you rewrite the whole code with it fixed?

Planning how to structure the rewritten code ›

Claude's response was interrupted.

also document the advanced techniques with docstrings please

Planning the file creation and design choices ›
I've rewritten the whole file with all the fixes and docstrings on the advanced techniques.

Created a file, shared files ›
I've rewritten the whole app in `construction_manager.py`, with the fixes from my review in place. I haven't run it, so test it in Drafter before you submit.

What changed

- **Messages show on the right page.** Every page now shows `state.message` through a small `take_message()` helper, which also clears it so it only appears once. Error messages now appear on the form you're filling out.
- **Duplicate project names are blocked** (the check ignores extra spaces around the name).
- **`selected_project` is now used.** It is set in `project_details`, `save_project`, `save_task` and `save_material`, and `add_task` and `add_material` use it as the dropdown default. It is now a real, changing state field, so you have 5 modified fields.
- **Number input no longer crashes.** The budget, quantity and price now come in as text, and `parse_number()` converts them. Bad input like "abc" or a blank box shows a friendly message.
- **Money is formatted** as `$1,234.50` through `format_money()`.
- **The unused `completion_percentage` field is removed.** Progress is always calculated from the tasks, so it can't get out of sync.
- I also added a `find_project()` helper and a `default_project_choice()` helper to remove repeated code.
- I removed the redundant progress-updating loops in `save_task` and `update_task_status`.

Advanced techniques, documented in docstrings

- `try/except` and `math.isfinite()` in `parse_number()`
- The `float | None` and `Project | None` return type hints
- The f-string format specifier `{value:,.2f}` in `format_money()`

- The `list[Project]` type hints in the `State` docstring

If your course never covered any of these, your instructor may want them documented exactly as they are here, so leave those docstrings in. The easiest way to avoid the documentation requirement entirely is to swap `try/except` for a simple check, but then bad input would go back to showing a Drafter error page.

One behavior to know about: if you click "Add a Task" from the main Tasks page, the dropdown defaults to the last project you viewed or added to, not necessarily the first project in the list.

is it possible to make it so each area for input is on a new line

Ran a command, shared files ›

Yes. I updated `construction_manager.py` so each input now sits on its own line. I added Drafter's `LineBreak()` right after every `TextBox`, `TextArea`, and `SelectBox` in the three forms (Add Project, Add Task, Add Material), 13 in total. The file compiles, but I couldn't run Drafter here, so please check how it looks in your browser.

Each label should now sit next to its input, like "Project Name: [____]", with the next field starting on a new line. If you'd rather have the label above its box, add a `LineBreak()` after each label string too.

If the spacing looks doubled or odd, your Drafter version may already add some spacing between components, and you can delete a few of the `LineBreak()` lines.

what about the \n

Checking whether HTML collapses newline characters ›

Using `"\n"` won't reliably work here. Drafter builds web pages, and in HTML a plain newline character in text gets collapsed into a space, so `"Project Name:\n"` would usually still show on the same line as the box that follows. `LineBreak()` produces an actual HTML line break, which is why I used it.

There are two places where `\n` does matter:

- **Inside a `TextArea`**, newlines are kept. If a user presses Enter in the Project Description box, the text keeps its line breaks in the saved data.
- **When displaying that saved text** (like the project description on the details page), the `\n` characters will probably collapse into spaces, so a multi-line description may show as one line. If that matters to you, I can split the description on `"\n"` and display each line separately. That would be an advanced technique, so I'd document it with a docstring like the others.

If you meant something else by "the \n", for example a specific spot in the code, tell me where and I'll look at it.

can i make it possible to remove materials

Ran a command, shared files ›

Yes, I added it to `construction_manager.py`. The file compiles, but I haven't run it in Drafter.

- **Remove button:** Each material on the Materials page now has a "Remove Material" button. To make that work, the loop now goes by position in the list, the same way your tasks page does.
- **New route:** `remove_material` checks that the position is valid, deletes that material with `pop()`, and shows a message like "Cement was removed." on the Materials page.
- **Budget updates automatically:** Material costs are calculated from the list each time, so the project's remaining budget and the budget summary update right after a removal.

Removal happens immediately, with no "are you sure?" step. If you want one, I can add a confirmation page. I can also add a remove button for tasks and projects, though removing a project would also need to delete its tasks and materials.

can you just send me the code i would need to paste in

Here are the two pieces to paste in for removing materials.

1. In the `materials` route, replace the `for material in state.materials:` loop with this:

python

```
    # Loop by position (index) so each Remove button knows which material
    # in the list it should delete.
    for material_number in range(len(state.materials)):
        material = state.materials[material_number]
        total_cost = material.amount * material.price

        content.append(Header(material.name, 2))
        content.append("Project: " + material.project_name)
        content.append(
            "Quantity: " + str(material.amount) + " " + material.unit
        )
        content.append("Price Per Unit: " + format_money(material.price))
        content.append("Total Cost: " + format_money(total_cost))
        content.append(
            Button(
                "Remove Material",
                "remove_material",
                [Argument("material_number", material_number)]
            )
        )
        content.append(HorizontalRule())
```

2. Add this new route anywhere above `start_server(...)`, for example right before the Budget Summary section:

python

```
@route
def remove_material(state: State, material_number: int) -> Page:
    # Make sure the position is valid before deleting anything.
    if material_number < 0 or material_number >= len(state.materials):
        state.message = "That material could not be found."
        return materials(state)

    removed = state.materials.pop(material_number)
    state.message = removed.name + " was removed."

    return materials(state)
```

from dataclasses import dataclass import math fr om drafter import * set_ website_title("constructi on-manager-site") set_si te_information(author...

PASTED

whats wrong with my code

Checking the materials route's missing return and button ›

Your `materials` route has two problems, both from where my loop snippet got pasted in.

1. Indentation error. The `for material_number in range(...)` line is indented 5 spaces instead of 4, so Python will refuse to run the file at all.

2. The end of the route is missing. My snippet replaced only the loop, but the original loop was followed by the Home button and the `return`. Those two lines got overwritten, so `materials` never returns a page and has no Home button.

Replace your whole `materials` route with this:

python

```
@route
def materials(state: State) -> Page:
    content = [
        Header("Construction Materials"),
        take_message(state),
        Button("Add a Material", "add_material"),
        HorizontalRule()
    ]

    if len(state.materials) == 0:
        content.append("No materials have been added yet.")

    # Loop by position (index) so each Remove button knows which material
    # in the list it should delete.
    for material_number in range(len(state.materials)):
        material = state.materials[material_number]
        total_cost = material.amount * material.price

        content.append(Header(material.name, 2))
        content.append("Project: " + material.project_name)
        content.append(
            "Quantity: " + str(material.amount) + " " + material.unit
        )
        content.append("Price Per Unit: " + format_money(material.price))
        content.append("Total Cost: " + format_money(total_cost))
        content.append(
            Button(
                "Remove Material",
                "remove_material",
                [Argument("material_number", material_number)]
            )
        )
        content.append(HorizontalRule())

    content.append(Button("Home", "home"))

    return Page(state, content)
```

I didn't spot any other bugs. Two smaller things:

- Your pasted file doesn't have the `LineBreak()` lines from the earlier "each input on a new line" change. If you still want those, tell me and I'll send just that snippet.

- `@route def add_material` and `@route def remove_material` sit right against the code above them. It works, but two blank lines before each one matches the rest of your file.

https://mhpust-cyber.github.io/construction-manager-site/?route=budget_summary

the thing up top says 1 error, do you knkow what it is

Fetched: Drafter App Server ›

I can't tell from here. The error banner is created by Drafter in your browser, and when I loaded the page I only got the home screen text, with no error message.

My best guess is that the deployed `construction_manager` file on GitHub still has the broken `materials` route (the extra indent and the missing `return`). Drafter checks every route when the site loads, so a route that never returns a page can trigger an error like that. If you haven't pushed the fixed `materials` route yet, do that first and reload.

If the error is still there, I need the actual message:

1. Look for the error text in the banner at the top, or in the debug panel under the page. If you can't see it, temporarily comment out `hide_debug_information()` near the top of your file, redeploy, and the full error should show.
2. Paste the message here, or a screenshot, and I can point to the line.

One more thing: a link with `?route=budget_summary` loaded fresh on GitHub Pages starts with an empty state. The summary page itself should still work, so the error most likely comes from the code rather than that link.

so does the website work?

Checking whether the site's app actually loads.

I can't say for sure. All I could see was the home page text, and I can't click through the site or see the error banner.

- **The code you pasted:** It won't run as-is. The extra space before the `for` line in `materials` is a syntax error, and Python refuses to load a file with one. Once you swap in the fixed `materials` route, I don't see anything else that should stop it.
- **The live site:** The home page text came through when I loaded it, so some version of your code is deployed and starting up. That could be an older version than the one you

pasted, and I can't tell which. The "1 error" banner means something is still wrong.

To find out where things stand:

1. Make sure the fixed `materials` route is in the file on GitHub and that the site has redeployed.
2. Reload the site, then click through Projects, Tasks, Materials, and Budget Summary.
3. If the error banner is still there, paste its text here and I'll find the cause.



Checking type hints for the in-browser runtime ›

Your code looks correct now: the `materials` route is fixed and I don't see any syntax errors. I still can't see the error banner's text, so what follows is a best guess rather than a confirmed diagnosis.

The most likely culprit is the `float | None` and `Project | None` return type hints. Sites on GitHub Pages run your Python in the browser using a smaller Python engine, and it may not support the `|` syntax in type hints. Python evaluates those hints when the function is defined, so an unsupported one could show up as an error right away.

You can test this by putting the hints in quotes, which makes Python treat them as plain text and never evaluate them. Change these three lines:

```python
def parse_number(text: str) -> "float | None":
```

```python
def find_project(state: State, project_name: str) -> "Project | None":
```

(`default_project_choice` has no `|`, so it needs no change.) The docstrings still document the technique, so you lose nothing.

If the error is still there after redeploying, please paste the exact text from the banner. If you can't read it, comment out `hide_debug_information()` temporarily so the full message shows. With that I can point to the exact line instead of guessing.

Two small cleanups, not errors: add two blank lines before `@route def remove_material` and before the `# BUDGET SUMMARY` comment, to match the rest of your file.

are the more advanced functions documented

Checking whether the remaining features appear in the docs ›

Mostly yes. These are documented in docstrings:

Technique	Where
`try/except`	`parse_number`
`float | None` return hint	`parse_number`
`math.isfinite()`	`parse_number`
f-string format spec `{value:,.2f}`	`format_money`
`Project | None` return hint	`find_project`
`list[Project]` etc. type hints	`State`

Three things aren't documented yet:

1. **`.pop()` in `remove_material`** has only a comment, not a docstring. Whether it counts as advanced depends on your course, but it's safer to document it.
2. **`list[str]` in `default_project_choice`** is a type hint that isn't explained there.
3. **If you apply my earlier suggestion to put the `|` hints in quotes**, the docstrings should mention that, because quoted hints are another thing a grader might ask about.

Here are the replacement pieces:

python

```
@route
def remove_material(state: State, material_number: int) -> Page:
    """Delete one material from the list and return to the Materials page.

    ADVANCED TECHNIQUE: list.pop(index) removes the item at that position
    in the list AND returns it. We keep the returned Material in
    "removed" so we can show its name in the confirmation message.
    The range check first makes sure the position exists, because pop()
    crashes with an IndexError if the position is out of range.
    """
    if material_number < 0 or material_number >= len(state.materials):
        state.message = "That material could not be found."
        return materials(state)

    removed = state.materials.pop(material_number)
    state.message = removed.name + " was removed."

    return materials(state)
```

python

```
def default_project_choice(state: State, project_options: list[str]) -> str:
    """Pick which project should be pre-selected in an "Add" form.

    ADVANCED TECHNIQUE: The type hint list[str] means project_options is
    a list that contains strings (the project names).

    Uses the project the user last viewed (state.selected_project) if it
    still exists; otherwise falls back to the first project.
    """
```

If you quote the hints, add this line to the docstrings of `parse_number` and `find_project`:

python

```
    The return type hint is written in quotes (for example "float | None")
    so Python treats it as plain text and does not evaluate it when the
    function is defined. This avoids errors on Python versions that do not
    support the | symbol in type hints.
```

If your course allows it, you could skip the extra documentation by removing the techniques themselves. Otherwise, with these additions every non-basic technique in the file is

explained.

Claude works directly with your codebase Install